SKYNYC · PROJECT ROADMAP

SkyNYC, Phase by Phase

Where the pipeline is, where it is going, and what each phase has to prove before the next one starts.

Seven phases · ~62 focused hours · cloud amendment v1.3 · updated August 2026

1. The destination

SkyNYC exists to answer one operational question with real evidence: **how much do arrival rates fall — and holding patterns and go-arounds rise — at JFK, LaGuardia, and Newark as weather degrades from clear conditions to instrument conditions, and which weather variable is the strongest leading indicator?**

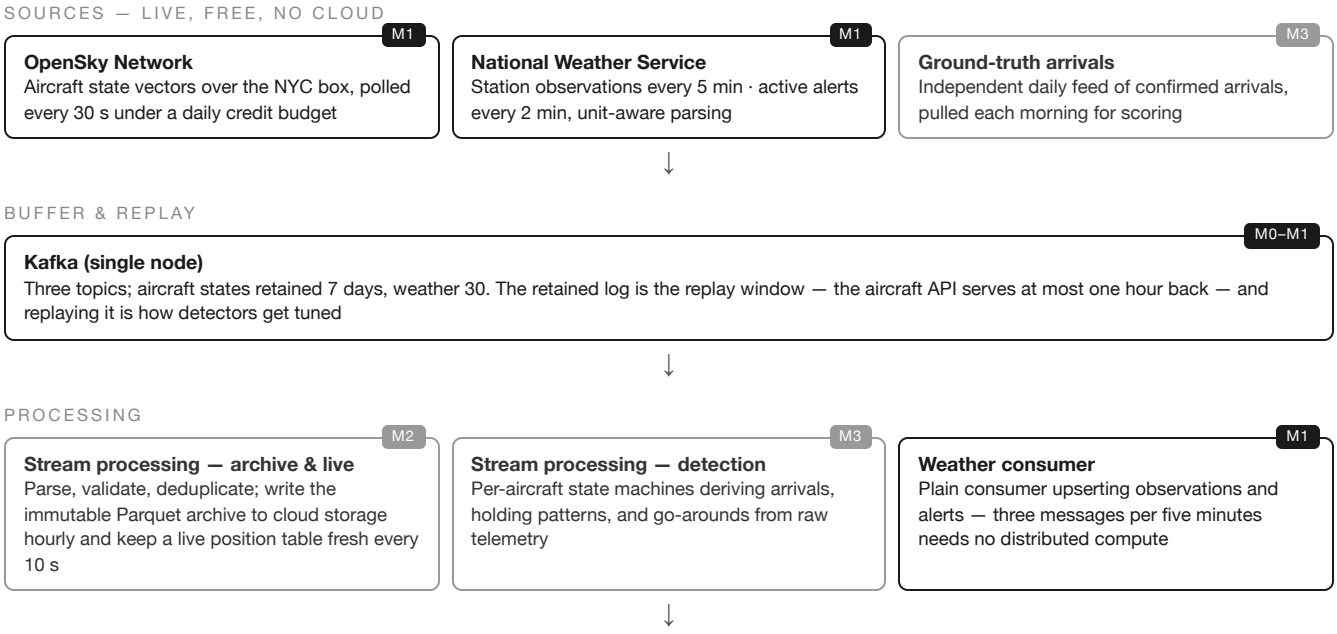
Four things must be true at the end for the project to count as finished:

1. A live dashboard — aircraft map, operational metrics, current weather — with end-to-end lag of two minutes or better at the 95th percentile.
2. An arrival detector scoring at least 85% precision and 80% recall against an independent ground-truth source, with the score published daily.
3. An analytical mart quantifying the arrival-rate delta by flight category over at least seven days of continuous operation.
4. Spend kept lean and legible — open-source software, free data sources, object storage that rounds to pennies, and one rented server as the only real line item.

The distinguishing work: the operational events do not exist in any feed. They are *derived* from raw aircraft telemetry by stateful stream processing, then *validated* against ground truth with a published quality score. That derive-and-validate loop is what separates this from an API-to-dashboard exercise.

2. The pipeline at a glance

Each box is tagged with the phase that builds it.



SERVING & ANALYTICS

M0
Postgres
Live positions, derived events, weather, ground truth — the serving layer and the warehouse

M4
dbt marts
Hourly airport facts, weather joined at read, the impact mart, and a daily detection-quality score — with tests and freshness checks that double as monitoring

M4
Batch scheduler
Daily ground-truth pull and hourly mart builds; streams stay supervised by Docker, not the scheduler

M2
Azure ADLS Gen2
The lake: immutable bronze archive + nightly database backups in object storage — compute is disposable, storage is not

M2, M5
Grafana dashboard
Live map, arrival rates, holding and go-arounds, conditions, alert overlays — provisioned from files in the repository

3. Where things stand

PHASE	DELIVERS	EST.	STATUS
M0 Foundations	Docker stack, schema, topics, live smoke test	4 h	**Complete**
M1 Ingestion	Both producers, weather consumer, credit guard, crash recovery, parser tests	8 h	**Complete**
M2 Live map	Streaming archive + live positions; first dashboard panels	10 h	YOU ARE HERE
M3 Detection & validation	Event detectors, ground-truth scoring, replay-based tuning	16 h	Planned
M4 Modeling	Full dbt project, tests, freshness, scheduled builds	10 h	Planned
M5 Dashboard & soak	All six panels, alert overlays, seven-day continuous run	8 h	Planned
M6 Analysis & packaging	The written answer, README, demo capture	6 h	Planned

> **WHY THE INGESTION PHASE RUNS CONTINUOUSLY FROM TODAY**
>
> Detector tuning in M3 works by replaying retained history with adjusted thresholds — and the retained Kafka log is the only history that exists. Every day the producers run now is a day of real traffic available for tuning later. Keep the pipeline collecting even while the next phases are being built; uptime today is tuning data tomorrow.

4. The phases in detail

M0 — Foundations 4 h · complete

Stand up the runtime: Kafka, Postgres with the schema applied on first boot, Grafana, the three topics with their retention policies, and a smoke test that proves authentication and both live data sources end to end. *Done meant:* the smoke test printed a valid token, a non-zero aircraft count over New York, and a station observation under two hours old — against the real APIs, no mocks.

M1 — Ingestion 8 h · complete

Turn two poll-only APIs into streams. The aircraft producer polls every 30 seconds inside a strict credit budget, degrades its own cadence when the budget runs low, backs off on upstream failures, and never dies on a bad poll. The weather producer polls observations and alerts, converts every value by its declared unit code, derives the flight category, and publishes only on change. A plain consumer upserts weather into Postgres with offsets committed only after the write lands. *Done meant:* messages flowing at the expected rate, deduplication observed, the credit projection under budget, and a hard-killed producer resuming on its own.

M2 — Live map 10 h · next up

The streaming layer earns its keep. One stream-processing application, two queries: an archive query writing validated, deduplicated Parquet partitioned by date and hour — the immutable system of record — and a live query keeping a current-position table fresh on a 10-second trigger. On top of it, the first visible product: the dashboard's live map and current-conditions panels, provisioned from files so the whole

dashboard is reproducible from a clone. *Done means:* aircraft visibly moving on the map, position staleness under a minute, archive partitions appearing hourly, and the stream surviving a broker restart unaided.

M3 — Detection & validation 16 h · the core

The hardest and most valuable phase. Per-aircraft state machines watch the position stream and derive the events no feed provides: arrivals (including the common case where coverage drops on short final), holding patterns (sustained turning without displacement, filtered to airliner classes), and go-arounds (descent, then a committed climb with no touchdown). Every detector ships with unit tests over recorded real sequences — positive and negative cases both. Then the validation loop: a daily pull of independent ground-truth arrivals, a scoring model computing precision and recall, and threshold tuning by replaying retained history rather than waiting for new traffic. *Done means:* at least 85% precision and 80% recall over a full day, published from the quality mart, with the fixture suite green.

M4 — Modeling 10 h

The analytical layer. Staging views over the raw facts; an hourly airport spine; weather joined to events *at read time* — the observation valid at each event's timestamp — which is the honest pattern when one stream changes every few seconds and the other a few times an hour. On top: the hourly airport fact table, the enriched events table, the weather-impact mart that answers the business question, and the daily detection-quality mart. Every model carries tests on its grain, and source freshness checks double as pipeline monitoring — freshness failing means ingestion is down, and that alarm working is part of the design. A scheduler takes over the daily ground-truth pull and hourly builds. *Done means:* the full build green including freshness, and scheduled runs green for 48 hours unattended.

M5 — Dashboard & soak 8 h

Finish the serving surface: arrival rates per airport, holding minutes and go-arounds, the active-alert timeline, and the arrival-rate-versus-conditions scatter colored by flight category — with weather alerts overlaid on the operational panels so cause and effect share an x-axis. Then the endurance test: seven continuous days of operation. *Done means:* seven unbroken days, 95th-percentile end-to-end lag of two minutes or better, zero unhandled crashes, and at least one real weather event captured in the data — in a New York August, the weather cooperates.

M6 — Analysis & packaging 6 h

Write the answer. The analysis document quantifies, with numbers from the impact mart, how arrival rates change from VFR through LIFR and which variable leads. The README gets the architecture, the quality scores, reproduction instructions — clone to running dashboard on a clean machine — and the required data-source citation. A short demo recording captures the storm-hits-metrics-move moment. *Done means:* every line of the definition of done above is checked, honestly.

5. Immediate next steps

#	ACTION	WHY NOW
1	Create the two cloud accounts and deploy — server + storage, both walked through in guide 3, ending with one deploy command	Moves collection to an always-on host; the laptop goes back to being a laptop
2	Stop the laptop's collector once the server is polling	One collector, one credit budget — two would double-spend it
3	Put the repository under version control with a first commit of the working M0+M1 state	The build history is part of the product; start it while the state is clean
4	Begin M2: the streaming application's archive and live queries, then the first dashboard panels	The next unbuilt boxes on the diagram — and the first visible payoff